

How models learn

Weights change only in training. Every other change lives in context, memory or state.

“Training changes the model. Inference uses it.”

Ask on every project

1. Does this need new behavior from the model, or just new facts?
2. How often does this information change, and who must be able to update it?
3. Should this change apply to one chat, one user or everyone?
4. Is this fact common, rare, private or newer than the model’s knowledge cutoff?
5. When the provider ships a new model version, what must we re-test or redo?

Rules of thumb

- Keep facts that change in a source of truth fed into context, never in the weights.
- Consider fine-tuning for consistent style, format or a narrow task, not to add facts.
- Treat the model’s recall of rare, private or recent facts as unreliable.
- Treat user feedback as logged data: it changes nothing until someone acts on it.

Red flags

- ⚠ “Users will correct it and it’ll learn over time”, with no pipeline behind it
- ⚠ A plan to fine-tune on docs so the model “knows” current policies
- ⚠ Answers about recent events with no search tool or retrieval behind them
- ⚠ Nobody can say where a wrong fact came from: weights, prompt or data

Key terms

Parameters	The numbers (weights) set by training. They don’t move while the model is used.
Post-training	Further training that turns a text predictor into an assistant with set behaviors.
Fine-tuning	Your own training run on your examples: a new model version to host and maintain.
Memory	Stored notes or history that the system re-sends as context on later requests.
Application state	What your product writes (orders, files, logs); it reaches the model only if sent.

DECISION GUIDE

Where does this change live?

If it matters for this request only	→ put it in the context.
If one user needs it across sessions	→ save it as memory, re-sent as context.
If it’s a fact that changes, like a price or policy	→ fix the source of truth and retrieve it.
If it records what happened, like an order or feedback	→ it’s application state.
If you need consistent behavior that prompts can’t get	→ consider a fine-tune, then evaluate.
If you need newer general knowledge in the weights	→ a new model version; until then, context.